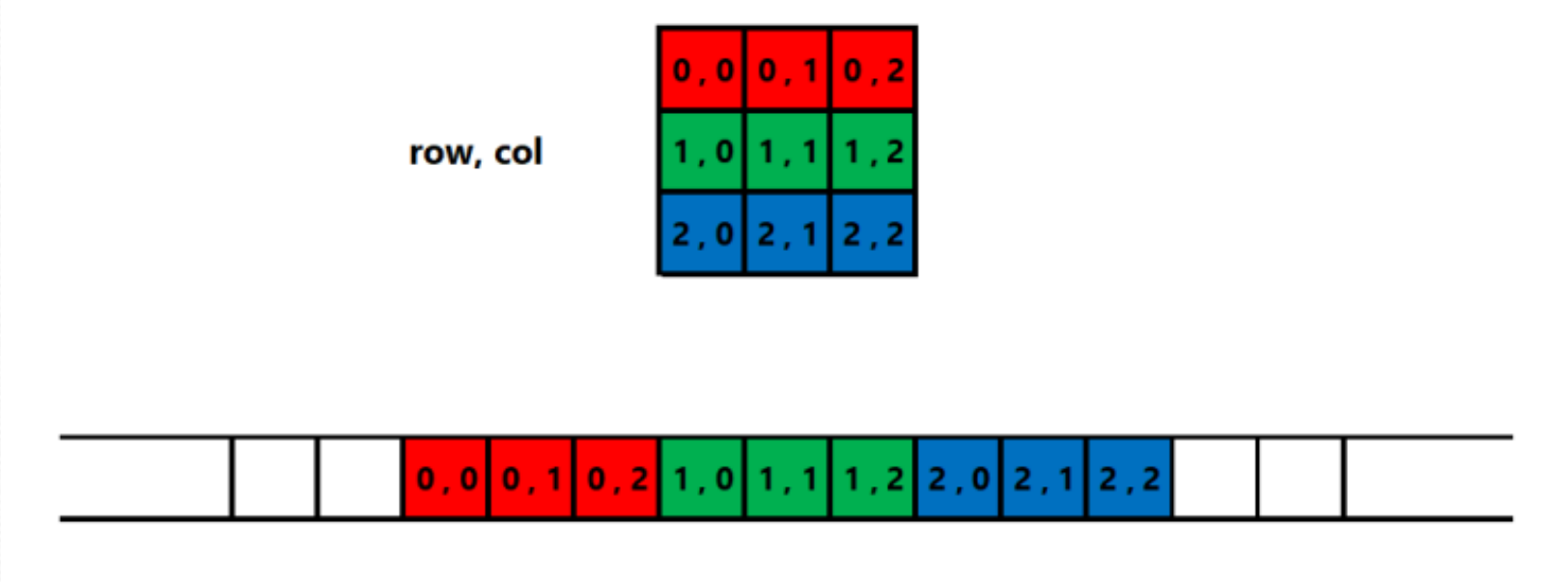
COAL

Lecture 11

2D Array

- A 2D array is a matrix of rows and columns.
- Stored in a memory as a linear array
- Can be accessed using base plus index addressing mode
- Base register (BX) will hold the base address of the array
- While the index address will be incremented as the linear array is accessed
  - [BX + SI]

2D Array



Linear Address Calculation

- linear_address = base_address + (row_index * num_columns + column_index) * element_size
- For (1,2), the linear address will be calculated by this equation.
- Please note that this address will be the offset part of logical address
- Whereas, the segment part will be from DS register.
  - If row_index = 1
  - num_columns = 3
  - column_index = 2
  - element_size = 1
- then, Linear_address = base_address + (1 * 3 + 2) * 1

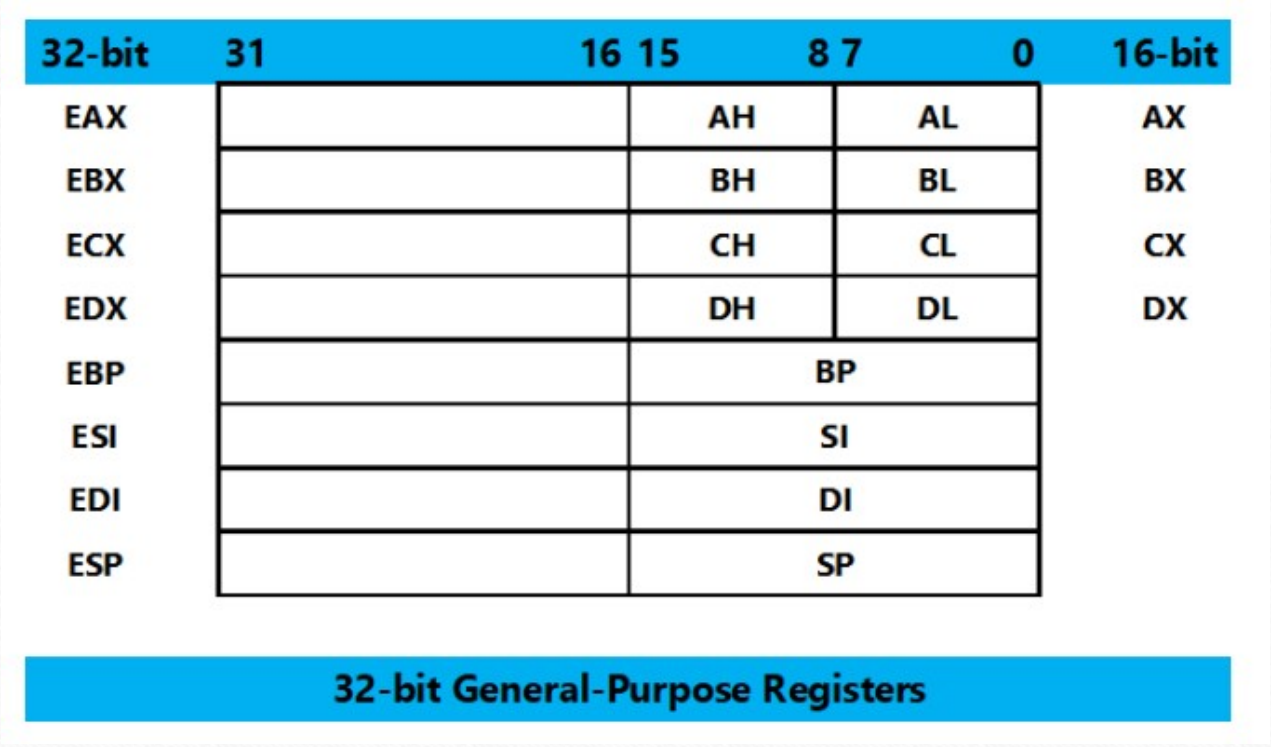
Example

```
.model small

.data

array2d db 1,2,3
        db 3,4,5
        db 6,9,8

.code

; Program to read index (2,1) of a 2D array

mov ax,@data
mov ds,ax

mov bx,offset array2d

mov al,3                    ; number of cols
mov dl,2                    ; mov row index to dl (i,j) = (2,1)
mov dh,1                    ; mov column index to dh (i,j) = (2,1)

mul dl                      ; multiple row index with number of col

add al,dh                   ; add col index to the product calculated in previous instruction

mov si,ax                   ; moving linear address to si

mov al,[bx+si]              ; reading desired element that is 9 at index (2,1)

.exit
```

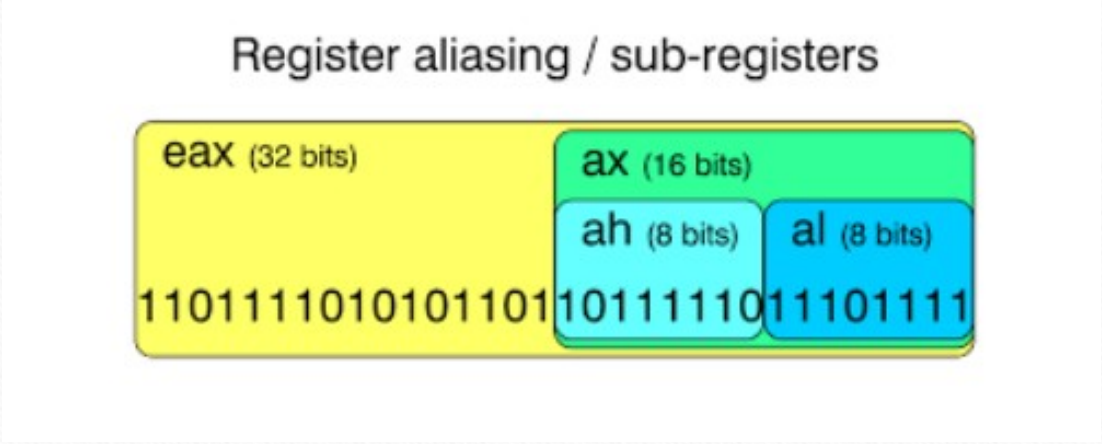
32-bit Registers

- Intel 80386 was the first 32-bit processor of the Intel family of microprocessors.
- It has 32-bit addresses and data buses.
- The protected mode was also introduced in this processor that limits the operation of user programs.
- It doesn’t allow user programs to go beyond their memory limits assign to them and directly access IO devices.
- The size of existing registers, except the segment registers, were extended to 32-bit.

32-bit Registers



Register Aliasing



32-bit Addressing Modes

- The addressing modes are the same.
- In 80386 and above, any 32-bit register may be used in register indirect addressing mode.
  - The MOV [EDX], CL instruction copies the byte sized contents of register CL into the data segment memory location addressed by EDX
- Any two registers from the above may be combined to generate the memory address.
  - The MOV [EAX+EBX], CL instruction copies the byte sized contents of register CL into the data segment memory location addressed by EAX plus EBX.
- Use any 32-bit register except ESP to address memory

32-bit Addressing Modes

- Register Direct Addressing (EAX, EBX)
- Register Indirect Addressing ([EBX], [EAX +4])
- Register Offset Addressing ([EBX + EAX], [EDX + ECX * 4])
- Base + Offset Addressing ([EBX + 10], [EAX + 0x20])
- Immediate Addressing (32-bit immediate value)
- Segment : Offset Addressing ([CS:1234], [DS:2000])

Inline Assembly Language

- Assembly language serves many purposes:
  - Improving program speed
  - Reducing memory needs
  - Controlling hardware when writing kernel modules
- You can use the inline assembler to embed assembly-language instructions directly in your C and C++ source programs without extra assembly and link steps.

Inline Assembly Language

- The inline assembler is built into the compiler, so you don't need a separate assembler
- Because the inline assembler doesn't require separate assembly and link steps, it is more convenient than a separate assembler
- Inline assembly code can use any C or C++ variable or function name that is in scope, so it is easy to integrate it with your program's C and C++ code.

__asm Keyword

- The__asm keyword invokes the inline assembler and can appear wherever a C or C++ statement is legal.
- It cannot appear by itself.
- It must be followed by an assembly instruction, a group of instructions enclosed in braces, or, at the very least, an empty pair of braces.

Example

```
__asm mov eax,ebx

or

__asm

{

Mov eax,ebx

}
```

Using and Preserving Registers in Inline Assembly

- A register will not have a given value when an __asm block begins.
- Register values are not guaranteed to be preserved across separate __asm blocks.
- If you end a block of inline code and begin another, you cannot rely on the registers in the second block to retain their values from the first block.

Example

Program to add two integer numbers using the inline assembly.

```
#include<iostream>
using namespace std;


int main(void)
{
        int a = 10, b = 20, c = 0;
        __asm
        {
                mov ebx,a
                add ebx,b
                mov c,ebx
        }
        cout << "Sum of " << a << " and " << b << " is " << c << endl;

}
```

Example – Sum of Array V1

```
#include <iostream>
using namespace std;

int array1[] = { 1,2,3,4,5 };
int sum = 0;
int main(void)
{
    __asm
    {
        push ebp
        push esi
        mov ebp, offset array1
        mov esi, 0
        mov ecx, 5
        mov eax, 0
        l1:
        add eax, dword ptr[ebp + esi]
            add esi, 4
            loop l1
            mov sum, eax
            pop esi
            pop ebp
    }
    cout << sum;
}
```

Example – Sum of Array V2

```
#include <iostream>
using namespace std;

int array1[] = { 1,2,3,4,5 };
int sum = 0;

int main(void)
{
    for (int i = 0; i < 20; i = i + 4)
    {
        __asm
        {
            push esi

            mov esi, i
            mov eax, dword ptr[array1 + esi]
            add sum, eax

            pop esi
        }
    }

    cout << sum;
}
```

Example – Rotate Left

```
#include<iostream>
using namespace std;
int main(void)
{
    int array1[] = { 1,2,3,4,5 };
    for (int i = 0; i < 20; i = i + 4)
    {
        __asm
        {
            mov eax, i
            rol dword ptr[array1 + eax], 1
        }
    }
    for (int i = 0; i < 5; i++)
    {
        cout << array1[i] <<
            endl;
    }
}
```

Example – 2D Array

```
#include<iostream>
using namespace std;
int main(void)
{
    int array1[3][4] = { {1,2,3,1},{4,5,6,1},{7,8,9,1} };
    int index = 0;
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 4; j++)
        {
            index = (i * 4 + j) * sizeof(int);
            __asm
            {
                mov eax, index
                add dword ptr[array1 + eax], 1
            }
            cout << "(" << i << "," << j << ")" << "->" << array1[i][j] << endl;
        }
}
```